
✓ ***Asignatura: Ciencia de Datos***

Unidad 1: Teoría - Introducción a Pandas

Universidad Nacional de Chilecito



Objetivos de la clase

- Aprender las bases de pandas: para qué sirve, cuáles son sus estructuras de datos y aplicaciones principales.
- Aplicar los conocimientos adquiridos de pandas en prácticas.

Pandas

-
- **Colección de estructuras de datos y funciones** que facilitan el trabajo con datos estructurados.
 - Nombre derivado de "**Panel Data System**" (término econométrico para datasets multidimensionales).
 - Construido **en base a Numpy**, inicialmente por Wes McKinney.
 - Ofrece más versatilidad que Numpy: podemos trabajar con **datos de diferentes tipos** y podemos identificar las filas y columnas **usando etiquetas**, en lugar de solamente enteros.

- Combina la alta performance de las **operaciones sobre arrays de NumPy con la flexibilidad en la manipulación de datos** de un spreadsheet o una base de datos relacional.
- Provee **funcionalidades de indexación avanzadas** para facilitar la manipulación, agregación y selección de partes de un dataset.
- Provee **operaciones de agrupación por columnas, filtros y sumalizaciones**.
- Estas características hacen de pandas una librería **indispensable para las tareas de limpieza y preprocesamiento de datos**.

✓ Estructuras de datos de Pandas

Veremos los siguientes objetos:

- Series
- DataFrames

✓ Series

- Una Series es un objeto similar a un vector unidimensional
- Contiene un array de valores y un array asociado de etiquetas de estos valores denominado como índice.
- Una Serie también puede ser pensada como un diccionario de tamaño fijo con sus claves ordenadas.
- Al igual que los arrays de NumPy, permite pasar una lista de valores con índices para seleccionar un subconjunto de valores.

	Animales	Name
0	Perro	
1	Oso	
2	Jirafa	
3	Tigre	
4	Serpiente	
5	Ratón	

Index

Values

En Python

```
import pandas as pd
```

```
lista = [0.25, 0.5, 0.75, 1.0]
data = pd.Series(lista)
data
```

```

      0
0  0.25
1  0.50
2  0.75
3  1.00

```

dtype: float64

```
data.values
```

```
array([0.25, 0.5 , 0.75, 1.  ])
```

```
data.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

▼ DataFrame

- Representa una estructura de datos tabular que contiene una colección de columnas, cada una de las cuales tiene un tipo determinado (number, string, boolean).
- El DataFrame tiene índices de columnas y filas.
- En el día a día de un data scientist la limpieza, preparado y normalización de los datos con los que trabaja es la tarea que más tiempo insume.
- Pandas es la librería principal al momento de tener que realizar estas tareas.
- Los DataFrame de Pandas son la herramienta fundamental que vamos a utilizar a lo largo del curso y de nuestro día de trabajo a partir de ahora.

The diagram illustrates a DataFrame structure. It features a table with two columns and six rows. The columns are labeled 'Animales' and 'Dueños'. The rows are indexed from 0 to 5. A vertical arrow on the left points downwards, labeled 'Eje 0 (filas)'. A horizontal arrow at the top points to the right, labeled 'Eje 1 (columnas)'. A red rectangular box highlights the entire table. Another red rectangular box highlights the row index '2' and the 'Jirafa' cell. A third red rectangular box highlights the 'Ratón' cell in the last row.

	Animales	Dueños
0	Perro	Juan
1	Oso	Pedro
2	Jirafa	Cristian
3	Tigre	Esteban
4	Serpiente	Pablo
5	Ratón	Claudio

▼ En Python

Vamos a construir dos instancias de Series:

- `area`, a partir de un diccionario
- `population`, a partir de dos listas

Luego construimos el DataFrame states a partir de las dos Series resultado del paso anterior

```
import pandas as pd
import numpy as np

area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,
             'Florida': 170312, 'Illinois': 149995}
area = pd.Series(area_dict)

states_list = ['Illinois', 'Texas', 'New York', 'Florida', 'California']
states_pop = [12882135, 26448193, 19651127, 19552860, 38332521]
population = pd.Series(states_pop, index= states_list)

states = pd.DataFrame({'population': population,
                       'area': area})
```

states

	population	area
California	38332521	423967
Florida	19552860	170312
Illinois	12882135	149995
New York	19651127	141297
Texas	26448193	695662

states.index

```
Index(['California', 'Florida', 'Illinois', 'New York', 'Texas'],
      dtype='object')
```

states.columns

```
Index(['population', 'area'], dtype='object')
```

Podemos seleccionar la columna 'area' del DataFrame

```
states['area']
```

```

      area
California 423967
Florida    170312
Illinois   149995
New York   141297
Texas      695662

dtype: int64
```

```
states.area
```

```

      area
California 423967
Florida    170312
Illinois   149995
New York   141297
Texas      695662

dtype: int64
```

Podemos indexar un DataFrame usando las posiciones (iloc) o los valores (loc)

```
states.iloc[:3, :1]
```

```

      population
California 38332521
Florida    19552860
Illinois   12882135
```

```
states.loc[:'Illinois', :'population']
```

```

      population
California 38332521
Florida    19552860
Illinois   12882135
```

Podemos indexar un DataFrame usando loc y máscaras booleanas

```
states.loc[states.area > 423000, ['population', 'area']]
```

	population	area
California	38332521	423967
Texas	26448193	695662

Podemos modificar valores de una DataFrame

```
states['density'] = states['population'] / states['area']
states
```

	population	area	density
California	38332521	423967	90.413926
Florida	19552860	170312	114.806121
Illinois	12882135	149995	85.883763
New York	19651127	141297	139.076746
Texas	26448193	695662	38.018740

✓ Práctica

✓ Ejercicio Series

Dadas dos series que representan cantidades

```
cantidades_prod1 = pd.Series([3, 7, 1], index=['local_a', 'local_b', 'local_c'])
```

```
cantidades_prod2 = pd.Series([54, 70, 985] index=['local_a', 'local_b', 'local_c'])
```

- ¿Qué cantidad de prod2 hay en local_b?
- Usando `iloc` obtener la suma de prod1 en local_a y local_c
- Usando `index` responder ¿qué local corresponde al índice 1 de cantidades_prod1?

```
import pandas as pd
import numpy as np
```

```
cantidades_prod1 = pd.Series([3, 7, 1], index=['local_a', 'local_b', 'local_
```

```
cantidades_prod2 = pd.Series([54, 70, 985], index=['local_a', 'local_b', 'lc
```

```
cantidades_prod1
```

	0
local_a	3
local_b	7
local_c	1

dtype: int64

```
cantidades_prod2
```

	0
local_a	54
local_b	70
local_c	985

dtype: int64

```
cantidad = cantidades_prod2.loc['local_b']  
cantidad
```

```
np.int64(70)
```

```
indices = [0, 2]  
cantidades_a_c = cantidades_prod1.iloc[indices]  
cantidades_a_c
```

	0
local_a	3
local_c	1

dtype: int64

```
np.sum(cantidades_a_c)
```

```
np.int64(4)
```

```
cantidades_prod1.index[1]
```

```
'local_b'
```

✓ Ejercicio DataFrame

- Construir una instancia de DataFrame usando las dos series del ejercicio anterior como columnas.
- Agregar al dataframe del paso anterior la columna 'total' cuyos datos sean el resultado de sumar las columnas 'cantidades_prod1' y 'cantidades_prod2'
- Mostrar los primeros dos registros de ese dataframe
- Mostrar los últimos dos registros de ese dataframe
- Obtener los nombres de locales que tengan en total más de 60 productos

```
data_dict = {'cantidades_prod1': cantidades_prod1, 'cantidades_prod2': canti  
data = pd.DataFrame(data_dict)  
data
```

```
data['total'] = data.cantidades_prod1 + data.cantidades_prod2  
data
```

```
data.head(2)
```

```
data.tail(2)
```

```
mask = data.total > 60  
data_mask = data.loc[mask, :]  
locales = data_mask.index  
locales.values
```

Referencias y Material Adicional